Question 7 — Jenkinsfile Pipeline Stages

Student: Kartik Roll No.: 2301730205

1. Aim

Define a declarative Jenkinsfile with five clearly separated pipeline stages (Checkout Build Test Package Deploy) and demonstrate that each stage executes successfully.

2. Complete Jenkinsfile

```
pipeline {
agent any

environment {
APP_DIR = '/opt/devops-lab1'
VENV_DIR = 'venv'
}

stages {

stage('Checkout') {
steps {
checkout scm
echo 'Source code checked out.'
}
}

stage('Build') {
steps {
sh '''
python3 -m venv ${VENV_DIR}
./${VENV_DIR}/bin/pip install --upgrade pip -q
./${VENV_DIR}/bin/pip install -r app/requirements.txt -q
echo "Build complete — Flask installed."
'''
}
}

stage('Test') {
steps {
sh '''
./${VENV_DIR}/bin/python -m pytest tests/ -v \
--tb=short --junitxml=test-results.xml
'''
}
post {
always {
junit 'test-results.xml'
```

```
}
}
}

stage('Package') {
steps {
sh '''
mkdir -p dist
tar -czf dist/app-build-${BUILD_NUMBER}.tar.gz app/
echo "Package created: dist/app-build-${BUILD_NUMBER}.tar.gz"
'''
archiveArtifacts artifacts: 'dist/*.tar.gz', fingerprint: true
}
}

stage('Deploy') {
steps {
sh 'sudo bash scripts/deploy.sh'
}
}
}

post {
success {
echo 'Pipeline completed successfully — application deployed.'
}
failure {
echo 'Pipeline FAILED — check the console output.'
}
}
}
```

3. Stage Descriptions

Stage	What it does	Key step
Checkout	Clones / updates the repository from GitHub	checkout scm
Build	Creates a Python virtualenv and installs Flask	pip install -r app/requirements.txt
Test	Runs pytest; publishes JUnit XML report	python -m pytest tests/ -v
Package	Creates a tar.gz archive and stores it as a Jenkins artefact	archiveArtifacts
Deploy	Runs deploy.sh — copies files, restarts systemd service, health-checks	sudo bash scripts/deploy.sh

4. Post Block

The post block at the end of the pipeline prints a success or failure message depending on the overall pipeline result. The Test stage's post block always publishes the JUnit XML file even when tests fail.

5. Execution Evidence

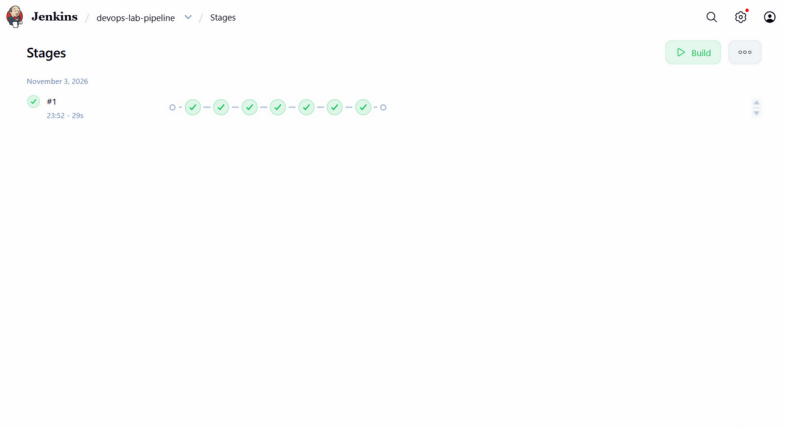


Fig: Build #1 — all five stages passed

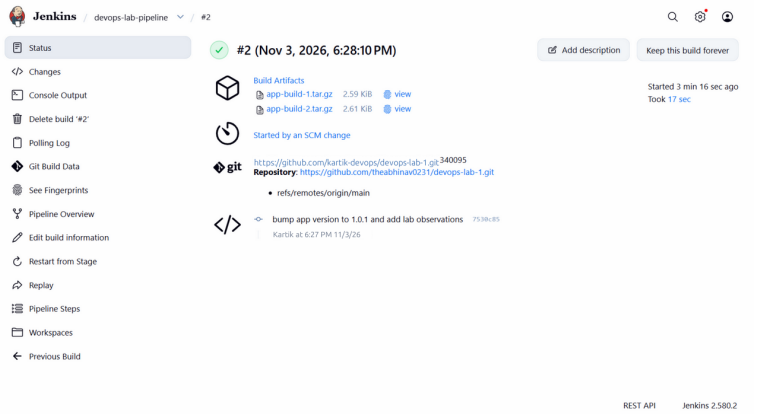


Fig: Build #2 — triggered by Poll SCM

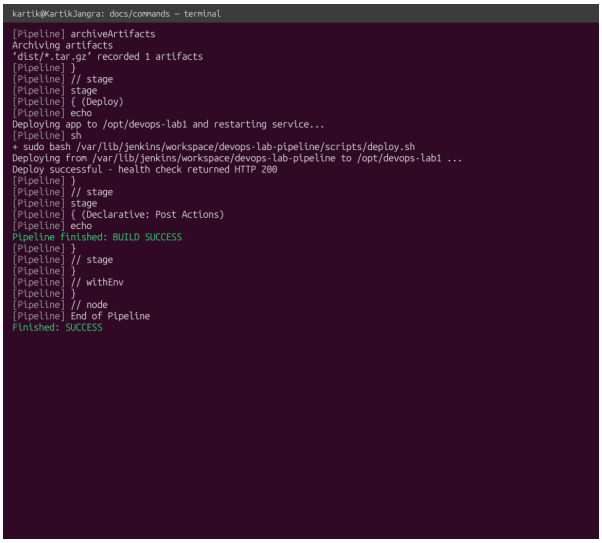


Fig: Console — pytest 2 tests passed

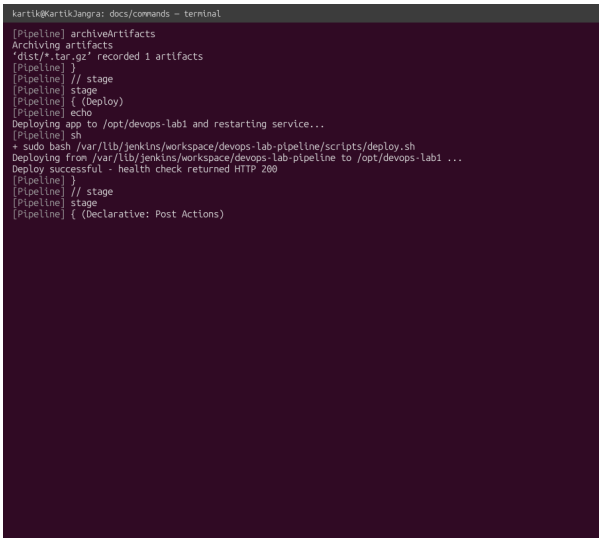


Fig: Console — Package and artefact archived

6. Observations

- The declarative syntax made the pipeline readable and easy to maintain.
- Each stage is isolated — a failure in Test stops Deploy from running.
- The Package stage archived the tar.gz so it is downloadable from the Jenkins UI.
- The post block ensured the JUnit report was published even on test failures.

7. Conclusion

The Jenkinsfile defined a complete, production-style CI/CD pipeline with five stages. The declarative syntax, post conditions and artefact archiving make the pipeline robust, traceable and ready for a real-world project.